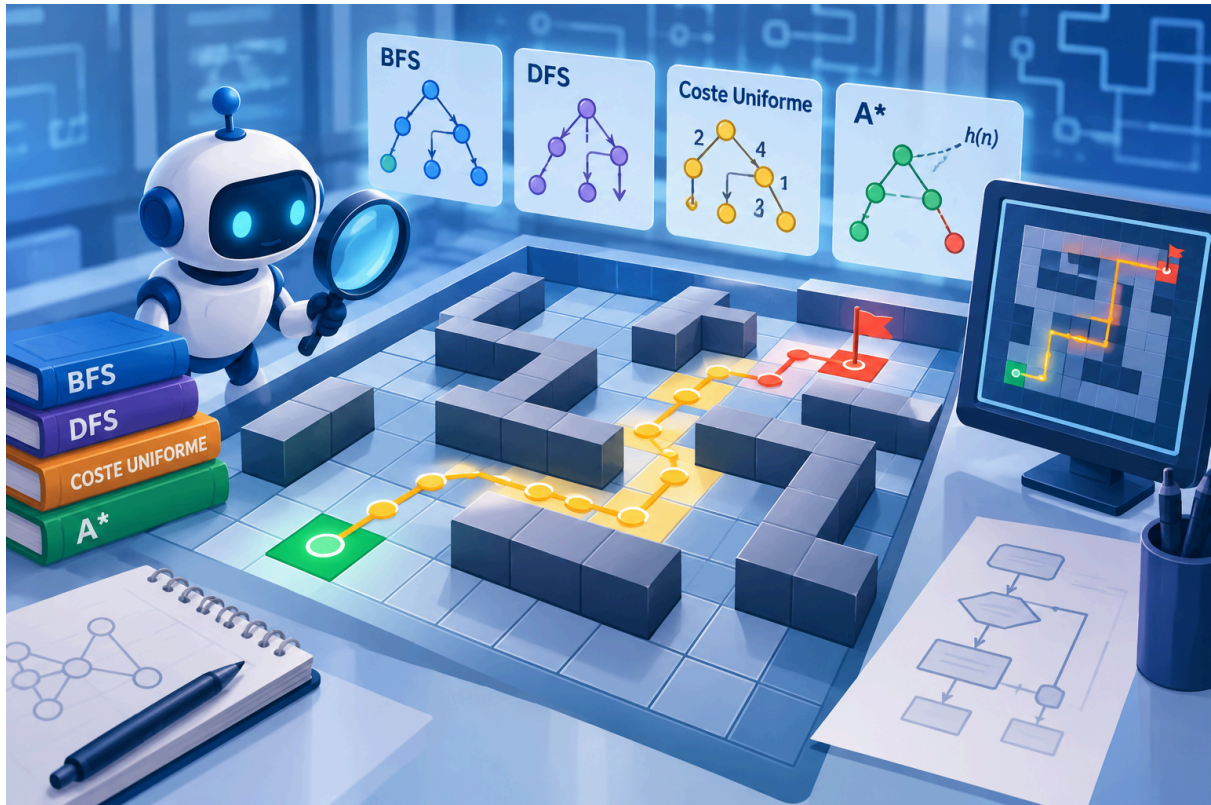


INFORME TÉCNICO



Laboratorio de rutas y algoritmos de búsqueda

Índice

1.Objetivo	3
2. Paso a paso de construcción	3
2.1. Definición del problema	3
2.2. Separación entre interfaz y algoritmos	3
2.3. Construcción del tablero	3
2.4. Dibujo	3
2.5. Implementación de los algoritmos	3
2.6. Animación	3
2.7. Pesos y costes	3
2.8. Escenarios y modo aleatorio	3
2.9. Comparación	3
2.10. Correcciones finales	4
3. Capturas de la aplicación	4
4. Arquitectura del proyecto	9
4.1. Estructura de datos del mapa	9
4.2. Flujo desde el clic hasta el resultado	9
5. Algoritmos de búsqueda	10
5.1. BFS — búsqueda en anchura	10
5.2. DFS — búsqueda en profundidad	10
5.3. Coste Uniforme — UCS	10
5.4. A*	10
6. Comparación de los cuatro escenarios	10
6.1. Conclusiones de la comparación	11
7. Lenguaje de etiquetado, estilos y programación	11
7.1. HTML: lenguaje de marcado	11
7.2. CSS: estilos y presentación	11
7.3. JavaScript: programación	11
8. Dificultades encontradas y correcciones	11
Pausa y limpieza	11
Scroll horizontal	12
Resultados	12
Pesos	12
Aleatorio	12

1.Objetivo

El objetivo del proyecto es entender y saber comparar 4 de los algoritmos que existen para poder aprender a tomar buenas decisiones en nuestros proyectos.

La aplicación permite colocar obstáculos, pesos, un inicio y varias metas. También incluye escenarios predefinidos y una opción aleatoria. Durante la ejecución se puede observar la exploración y comparar el número de nodos explorados, los pasos del camino y su coste.

2. Paso a paso de construcción

2.1. Definición del problema

El laberinto es una cuadrícula de 20×20 . Cada celda puede estar vacía, ser un obstáculo, tener un peso, ser el inicio o ser una meta.

2.2. Separación entre interfaz y algoritmos

Separamos los archivos javascript del HTML. algorithms.js contiene los 4 algoritmos y app.js se encarga de la funcionalidad, llamando a cada algoritmo en su momento.

2.3. Construcción del tablero

HTML define las zonas principales y JavaScript crea dinámicamente las 400 celdas. CSS Grid permite colocar las celdas de forma ordenada y el diseño responsive adapta su tamaño.

2.4. Dibujo

Se añaden herramientas para obstáculo, borrar, inicio, meta y peso.

2.5. Implementación de los algoritmos

Cada algoritmo recibe la misma cuadrícula, el mismo inicio y la meta activa. Así se pueden comparar bajo las mismas condiciones.

2.6. Animación

Después de calcular la solución, app.js recorre la lista de celdas exploradas y después la ruta final. El control de pausa utiliza una espera que se libera al continuar.

2.7. Pesos y costes

Las celdas con peso tienen un coste de entrada. Coste Uniforme y A* lo utilizan. Las metas pueden tener además un coste adicional.

2.8. Escenarios y modo aleatorio

Se crean los cuatro escenarios para estudiar situaciones distintas. El botón Aleatorio genera inicio, metas, obstáculos y pesos en posiciones aleatorias.

2.9. Comparación

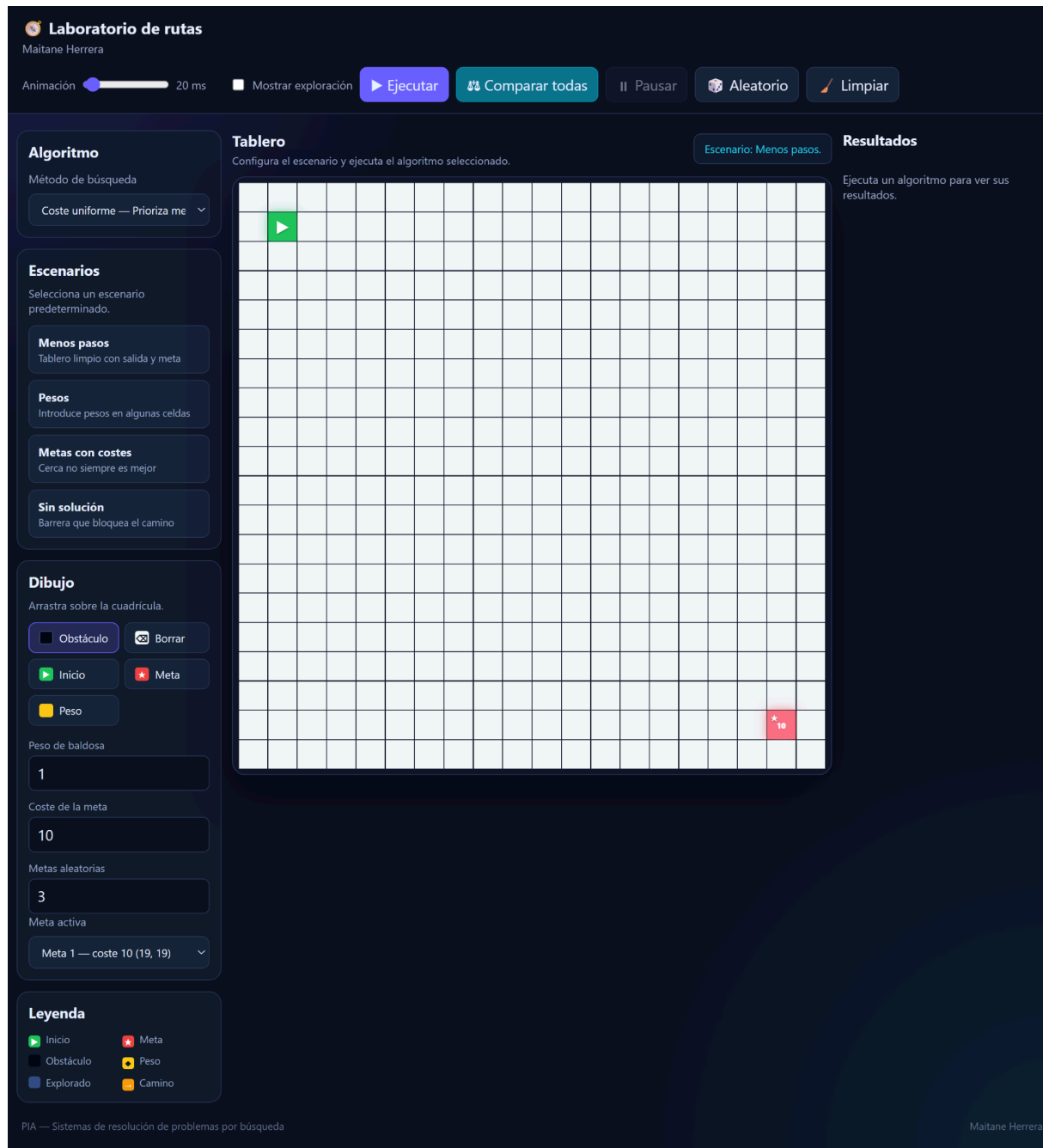
Comparar todas ejecuta los cuatro algoritmos sobre la meta activa y presenta una tabla con características y métricas.

2.10. Correcciones finales

Durante el desarrollo se han ido adaptando tanto el index.html como app.js, se hizo responsive la cuadrícula, se evitó el scroll horizontal y se adaptó la columna de resultados.

3. Capturas de la aplicación

Imagen predeterminada al acceder



Laboratorio de rutas

Maitane Herrera

Animación

20 ms

Mostrar exploración

Ejecutar

Comparar todas

Pausar

Aleatorio

Limpiar

Algoritmo

Método de búsqueda

BFS — Búsqueda en anchura

Escenarios

Selecciona un escenario predeterminado.

Menos pasos

Tablero limpio con salida y meta

Pesos

Introduce pesos en algunas celdas

Metas con costes

Cerca no siempre es mejor

Sin solución

Barrera que bloquea el camino

Dibujo

Arrastra sobre la cuadrícula.

Obstáculo

Borrar

Inicio

Meta

Peso

Peso de baldosa

1

Coste de la meta

10

Metas aleatorias

3

Meta activa

Meta 1 — coste 10 (12, 11)

Inicio

Obstáculo

Explorado

Meta

Peso

Camino

Tablero

Configura el escenario y ejecuta el algoritmo seleccionado.

Comparación terminada.

Comparación de algoritmos

Resultados de BFS, DFS, Coste Uniforme y A* sobre el tablero actual.

Meta activa: fila 12, columna 11 · coste 10

Algoritmo	Tipo de búsqueda	Pesos	Heurística	Encontrado	Nodos explorados	Pasos	Coste
BFS	Anchura	No	No	Sí	52	6	6
DFS	Profundidad	No	No	Sí	104	78	78
Coste uniforme	Coste mínimo	Sí	No	Sí	47	6	6
A*	Coste + heurística	Sí	Sí	Sí	13	6	6

Resultados

BFS

Encontrado

Explorados

Pasos

Coste

DFS

Encontrado

Explorados

Pasos

Coste

Coste uniforme

Encontrado

Explorados

Pasos

Coste

A*

Encontrado

Explorados

Pasos

Coste

PIA — Sistemas de resolución de problemas por búsqueda

Maitane Herrera

Laboratorio de rutas

Maitane Herrera

Animación

20 ms

☒Mostrar exploración

Ejecutar

Comparar todas

⏸Pausar

Aleatorio

Limpiar

Algoritmo

Método de búsqueda

Coste uniforme — Prioriza me

Escenarios

Selecciona un escenario predeterminado.

Menos pasos

Tablero limpio con salida y meta

Pesos

Introduce pesos en algunas celdas

Metas con costes

Cerca no siempre es mejor

Sin solución

Barrera que bloquea el camino

Dibujo

Arrastra sobre la cuadrícula.

☐Obstáculo

Borrar

Inicio

Meta

Peso

Peso de baldosa

1

Coste de la meta

10

Metas aleatorias

3

Meta activa

Meta 1 — coste 50 (5, 18)

Leyenda

Inicio

Meta

☐Obstáculo

Peso

Explorado

Camino

Tablero

Configura el escenario y ejecuta el algoritmo seleccionado.

Coste uniforme ha encontrado la meta.

Comparación de algoritmos

Resultados de BFS, DFS, Coste Uniforme y A* sobre el tablero actual.

Meta activa: fila 5, columna 18 · coste 50

Algoritmo	Tipo de búsqueda	Pesos	Heurística	Encontrado	Nodos explorados	Pasos	Coste
BFS	Anchura	No	No	Sí	231	19	19
DFS	Profundidad	No	No	Sí	56	55	55
Coste uniforme	Coste mínimo	Sí	No	Sí	231	19	19
A*	Coste + heurística	Sí	Sí	Sí	68	19	19

Resultados

Coste uniforme

Encontrado

Explorados

Pasos

Coste

231

19

19

50

PIA — Sistemas de resolución de problemas por búsqueda

Maitane Herrera

Laboratorio de rutas

Maitane Herrera

Animación

20 ms

Mostrar exploración

Ejecutar

Comparar todas

Pausar

Aleatorio

Limpiar

Algoritmo

Método de búsqueda

BFS — Búsqueda en anchura

Escenarios

Selecciona un escenario predeterminado.

Menos pasos

Tablero limpio con salida y meta

Pesos

Introduce pesos en algunas celdas

Metas con costes

Cerca no siempre es mejor

Sin solución

Barrera que bloquea el camino

Dibujo

Arrastra sobre la cuadrícula.

Obstáculo

Borrar

Inicio

Meta

Peso

Peso de baldosa

1

Coste de la meta

10

Metas aleatorias

3

Meta activa

Meta 1 — coste 10 (19, 19)

Legenda

Inicio

Obstáculo

Explorado

Meta

Peso

Camino

Tablero

Configura el escenario y ejecuta el algoritmo seleccionado.

BFS ha encontrado la meta.

Comparación de algoritmos

Resultados de BFS, DFS, Coste Uniforme y A* sobre el tablero actual.

Meta activa: fila 19, columna 19 · coste 10

Algoritmo	Tipo de búsqueda	Pesos	Heurística	Encontrado	Nodos explorados	Pasos	Coste
BFS	Anchura	No	No	Sí	396	34	34
DFS	Profundidad	No	No	Sí	364	38	38
Coste uniforme	Coste mínimo	Sí	No	Sí	393	34	34
A*	Coste + heurística	Sí	Sí	Sí	202	34	34

Resultados

BFS

Encontrado

Explorados

Pasos

Coste

396

34

34

34

PIA — Sistemas de resolución de problemas por búsqueda

Maitane Herrera

Laboratorio de rutas
Maitane Herrera

Animación 20 ms
☒ Mostrar exploración

Ejecutar

Comparar todas

⏸ Pausar

🎲 Aleatorio

🧹 Limpiar

Algoritmo
Método de búsqueda

BFS — Búsqueda en anchura

Escenarios
Selecciona un escenario predeterminado.

Menos pasos
Tablero limpio con salida y meta

Pesos
Introduce pesos en algunas celdas

Metas con costes
Cerca no siempre es mejor

Sin solución
Barrera que bloquea el camino

Dibujo
Arrastra sobre la cuadrícula.

■ Obstáculo

✖ Borrar

▶ Inicio

★ Meta

🟡 Peso

Peso de baldosa
1

Coste de la meta
10

Metas aleatorias
3

Meta activa
Meta 1 — coste 10 (18, 11)

Leyenda

▶ Inicio

★ Meta

■ Obstáculo

🟡 Peso

■ Explorado

🟡 Camino

Tablero
Configura el escenario y ejecuta el algoritmo seleccionado.

BFS no ha encontrado un camino.

Comparación de algoritmos
Resultados de BFS, DFS, Coste Uniforme y A* sobre el tablero actual.

Meta activa: fila 18, columna 11 - coste 10

Algoritmo	Tipo de búsqueda	Pesos	Heurística	Encontrado	Nodos explorados	Pasos	Coste
BFS	Anchura	No	No	No	200	—	—
DFS	Profundidad	No	No	No	200	—	—
Coste uniforme	Coste mínimo	Sí	No	No	200	—	—
A*	Coste + heurística	Sí	Sí	No	200	—	—

Resultados

BFS
Encontrado
Explorados
Pasos
Coste

No
200
—
—

PIA — Sistemas de resolución de problemas por búsqueda
Maitane Herrera

4. Arquitectura del proyecto

La aplicación utiliza una arquitectura sencilla de tres capas: estructura, presentación y lógica.

Archivo	Responsabilidad	Ejemplos
index.html	Estructura y contenido	Barra de controles, panel izquierdo, tablero, resultados y tabla comparativa.
style.css	Presentación visual	Colores, celdas, botones, leyenda, tabla y diseño responsive.
app.js	Lógica de aplicación	Eventos, dibujo, tableros, animación, pausa, comparación y actualización de resultados.
algorithms.js	Algoritmos de búsqueda	BFS, DFS, Coste Uniforme, A*, vecinos, costes y reconstrucción de ruta.
README.md	Documentación	Objetivo, tecnologías, instrucciones, algoritmos, escenarios, pruebas y limitaciones.

4.1. Estructura de datos del mapa

Cada posición de la cuadrícula se representa mediante un objeto con información de su estado:

```
{
  row: número de fila,
  col: número de columna,
  wall: true/false,
  weight: coste de entrada,
  goalCost: coste adicional de la meta
}
```

4.2. Flujo desde el clic hasta el resultado

1. El usuario pulsa o arrastra sobre una celda.
2. app.js identifica la herramienta seleccionada.
3. applyTool modifica los datos de la celda posición de inicio/meta.
4. renderGrid vuelve a dibujar el tablero.
5. Al pulsar Ejecutar, se obtiene el algoritmo y la meta activa.
6. executeAlgorithm pasa una copia del inicio y la meta al algoritmo.
7. El algoritmo explora la cuadrícula y devuelve found, explored, path y cost.
8. animateResult muestra la exploración y después la ruta.
9. showResults y la tabla comparativa muestran las métricas.

5. Algoritmos de búsqueda

5.1. BFS — búsqueda en anchura

BFS utiliza una cola. Primero entra el inicio y después sus vecinos. Los vecinos de esos vecinos se añaden detrás. Por eso explora por niveles.

5.2. DFS — búsqueda en profundidad

DFS utiliza una pila. El algoritmo continúa por una rama hasta que no puede seguir y entonces retrocede.

5.3. Coste Uniforme — UCS

UCS selecciona la celda cuyo coste acumulado desde el inicio es menor.

5.4. A*

A* combina el coste ya recorrido con una estimación del coste que falta.

6. Comparación de los cuatro escenarios

Las cifras de esta tabla son una ejecución representativa. En el escenario Pesos, las posiciones y valores se generan aleatoriamente, por lo que los resultados cambian entre ejecuciones. En Metas con costes se ha usado la primera meta como meta activa, tal como aparece inicialmente en la aplicación.

Escenario	Algoritmo	Encontrado	Exploradas	Pasos	Coste
Menos pasos	BFS	Sí	396	34	34
Menos pasos	DFS	Sí	364	38	38
Menos pasos	UCS	Sí	396	34	34
Menos pasos	A*	Sí	324	34	34
Pesos (ejemplo)	BFS	Sí	396	34	34
Pesos (ejemplo)	DFS	Sí	364	38	38
Pesos (ejemplo)	UCS	Sí	394	34	34
Pesos (ejemplo)	A*	Sí	218	34	34
Metas con costes	BFS	Sí	231	19	19
Metas con costes	DFS	Sí	56	55	55
Metas con costes	UCS	Sí	231	19	69
Metas con costes	A*	Sí	68	19	69
Sin solución	BFS	No	200	—	—
Sin solución	DFS	No	200	—	—
Sin solución	UCS	No	200	—	—
Sin solución	A*	No	200	—	—

6.1. Conclusiones de la comparación

- Menos pasos: BFS, UCS y A* encuentran la ruta de 34 pasos en este tablero libre. DFS encuentra una ruta más larga, lo que demuestra que no garantiza el mínimo número de movimientos.
- Pesos: UCS y A* tienen en cuenta el coste de las celdas. A* puede reducir bastante la exploración gracias a la heurística. Las cifras exactas dependen de los pesos aleatorios.
- Metas con costes: el coste de la meta puede cambiar el coste final de UCS y A*. Por eso una ruta geométricamente corta no tiene por qué ser la opción más barata si la meta tiene un coste adicional alto.
- Sin solución: los cuatro algoritmos terminan sin encontrar una ruta porque la barrera divide el tablero.

7. Lenguaje de etiquetado, estilos y programación

7.1. HTML: lenguaje de marcado

Es un lenguaje de marcado que define qué elementos existen en la página y cómo se organizan.

Las secciones más importantes son: header para la cabecera y controles; aside para el panel de herramientas y resultados; main para el espacio de trabajo; section para agrupar tablero y comparación; button para las acciones; input/select para parámetros; table para la comparación.

7.2. CSS: estilos y presentación

CSS define cómo se ven los elementos. Controla colores, tamaños, márgenes, tipografía, bordes, distribución y comportamiento responsive.

CSS Grid es especialmente importante porque permite organizar la cuadrícula y adaptar el tamaño de las celdas. También se utilizan media queries para que la aplicación se adapte a pantallas pequeñas y evitar scroll horizontal.

7.3. JavaScript: programación

JavaScript gestiona eventos, modifica los datos del tablero, ejecuta los algoritmos, anima la exploración y actualiza los resultados.

8. Dificultades encontradas y correcciones

Pausa y limpieza

Limpiar podía quedar condicionado por una animación pausada. Hago que aunque esté la animación en pausa se pueda limpiar el tablero.

Scroll horizontal

El tablero tenía un tamaño fijo que podía superar el espacio disponible. Se hizo responsive y las celdas se adaptan al contenedor.

Resultados

La información se trasladó a una columna derecha y se creó una tabla comparativa debajo del tablero.

Pesos

Se hace que el escenario Pesos coloque posiciones y valores aleatorios y que el dibujo manual pinte la celda en amarillo mostrando su valor.

Aleatorio

El botón genera inicio, metas, obstáculos y pesos de forma aleatoria.